Solar Intelligence System

Technical Documentation & Operations Manual

Version 8.0 — April 2026

Hinode SOT/SP Spectropolarimetric Analysis Pipeline

Confidential — For Authorized Personnel Only

DOCUMENT 1 OF 2

Standalone Version Manual WINDOWS X64

For operators, analysts, and non-developer users.

1. Overview

The **Solar Intelligence System (SIS)** is a desktop application designed to analyze spectropolarimetric data from the **Hinode** solar observation spacecraft. In plain terms, it takes raw telescope data files and automatically extracts key physical measurements about the Sun's magnetic field, surface temperature, plasma velocity, and turbulence — measurements that would normally take a trained astrophysicist hours to produce manually.

The standalone version packages the entire system into a single Windows executable. No Python installation, no terminal commands, no dependency management. You extract a folder, double-click an icon, and start analyzing data.

What it produces:

- A CSV spreadsheet with per-pixel magnetic field and plasma measurements
- A high-resolution **Master Dashboard** combining all key visualizations into one image
- Detailed diagnostic plots for every analyzed pixel (Stokes I, Q, U, V profiles)
- An AI-generated scientific interpretation report (powered by Google Gemini)

2. System Requirements

COMPONENT	MINIMUM	RECOMMENDED
Operating System	Windows 10 (64-bit)	Windows 11 (64-bit)
RAM	4 GB	8 GB or more
Disk Space	600 MB (application)	2 GB (including outputs)
Internet	Optional (AI report only)	Broadband connection

NOTE

The AI Scientific Report feature requires an active internet connection to reach the Google Gemini API. All other features (classification, physics analysis, dashboard generation) work fully offline.

3. Installation and Startup

Step 1 — Download and Extract

1. Download the file `SIS_v8_Windows_x64.zip`.
2. Right-click the ZIP file and select **"Extract All..."** (or use 7-Zip / WinRAR).
3. Choose a destination folder (e.g., your Desktop or Documents).

After extraction, you should see a folder with this structure:

```
SIS_v8_Windows_x64/
├── sis_gui_app_v2.exe   ← The main application
├── xgb_model.pkl        ← Pre-trained AI classifier
└── _internal/           ← Runtime libraries (DO NOT DELETE)
    └── (~5,000 support files)
```

CRITICAL

The `_internal` folder, `xgb_model.pkl`, and the `.exe` file must all remain **in the same folder**. Moving the `.exe` to your Desktop without the rest of the folder will cause the application to crash immediately.

Step 2 — Launch

Double-click `sis_gui_app_v2.exe`. The application window will appear within a few seconds.

Step 3 — Windows SmartScreen Warning (First Launch Only)

Because this application is not signed with a commercial code-signing certificate, Windows may show a blue "Windows protected your PC" dialog.

How to proceed:

1. Click "**More info**" (the small text link below the warning).
2. Click "**Run anyway**".

This only happens once. The application is built from open-source scientific Python libraries using PyInstaller.

4. Using the Application

4.1 The Control Center (Tab 1)

This is your main workspace. Here you will:

- **Name your session:** Enter a descriptive name (e.g., `Active_Region_12345`). If you leave it blank, the system auto-generates a timestamped name.
- **Select your FITS file:** Click "Select FITS File" and browse to your Hinode SOT/SP `.fits` data file.
- **Launch the analysis:** Click the large blue "**LAUNCH SIS ANALYSIS**" button.

The **Live Console** panel shows real-time progress from the backend engine. The progress bar animates while processing is active. **Do not close the window** during analysis.

4.2 The Master Dashboard (Tab 2)

When analysis completes, the application automatically switches to this tab and displays the five-panel **Master Dashboard**:

PANEL	WHAT IT SHOWS
A — Magnetic Vector	LOS and transverse magnetic field components along the scan
B — Doppler Velocity	Plasma line-of-sight velocity (blue = approaching, red = receding)
C — Turbulence	Non-thermal broadening across the scan
D — AI Routing	Pie chart showing how the AI classified and routed each pixel
E — Temperature Map	2D heatmap of brightness temperature from deep photosphere to upper layers

4.3 The AI Report (Tab 3)

If a Gemini API key is configured, this tab displays a professionally written scientific interpretation generated by Google's Gemini 2.5 Flash model.

4.4 The Repository (Tab 4)

A built-in file explorer rooted at the `SIS_Runs/` workspace folder. Every analysis session creates its own subfolder here.

5. How the System Works

Phase 1 — The XGBoost AI Triage

Think of this as a **smart traffic cop**. Before any expensive physics calculations begin, a pre-trained machine learning model (XGBoost) examines every pixel in your dataset and instantly sorts it into one of three categories:

- **"Clear"** — Strong, clean magnetic signal detected. Route to the precision Zeeman analysis engine.
- **"Noisy"** — Weak or ambiguous signal. Route to the gentler Weak Field Approximation method.
- **"Anomaly"** — Something physically unusual. Flag it and archive it for human review.

This triage takes **milliseconds** for the entire dataset. The XGBoost model is stored in `xgb_model.pkl`. If the file is missing, the system falls back to a built-in heuristic classifier.

Phase 2 — The Physics Engine

Based on the AI triage, each pixel is routed to the appropriate physics solver. The system computes the "Fantastic Four" measurements:

1. **Magnetic Field Strength** — How strong is the Sun's magnetic field at this point? (Gauss)
2. **Temperature Profile** — Temperature from the deep photosphere to the upper layers (Kelvin)
3. **Plasma Velocity** — How fast is the solar plasma moving toward or away from us? (km/s)
4. **Turbulence** — Chaotic plasma motion beyond what temperature alone would predict (km/s)

Phase 3 — The AI Reporter

After all physics calculations are complete, SIS sends a statistical summary to Google Gemini, which generates a professional scientific interpretation report.

6. Output Files

All outputs are saved inside `SIS_Runs/<Session_Name>/`:

FILE	DESCRIPTION
`*_results.csv`	Full per-pixel results spreadsheet
`*_SIS_v3_Master_Dashboard.png`	5-panel visualization dashboard
`*_AI_SCIENTIFIC_REPORT.txt`	AI-generated scientific report
`*_B_profile.png`	Magnetic field strength along the scan
`*_LOS_velocity_COG.png`	Doppler velocity map
`*_turbulence_profile.png`	Turbulence velocity profile
`*_temperature_stratification.png`	2D temperature heatmap
`*_magnetic_vector_overview.png`	Combined LOS + transverse B-field
`*_examples/`	Per-pixel 4-panel diagnostic plots
`*_anomalies/`	Archived anomalous Stokes profiles (.npy)

7. Troubleshooting

"The application won't start / crashes immediately"

Cause: The `.exe` was moved away from the `_internal` folder.

Fix: Move `sis_gui_app_v2.exe` back into the `SIS_v8_Windows_x64` folder so it sits next to `_internal` and `xgb_model.pkl`.

"Windows SmartScreen blocks the application"

Cause: The executable is not digitally signed.

Fix: Click "More info" then "Run anyway". This is expected behavior for unsigned scientific software.

"Antivirus quarantines the EXE"

Cause: Some antivirus programs flag PyInstaller-built executables as suspicious.

Fix: Add the `SIS_v8_Windows_x64` folder to your antivirus exclusion list.

"The AI Report tab is empty"

Cause: The Gemini API key is not configured, or there is no internet connection.

Fix: This is not critical. All physics results, the CSV, and the dashboard are generated locally and are unaffected.

"XGBoost model not found — Using heuristic fallback"

Cause: The file `xgb_model.pkl` is missing from the application folder.

Fix: Ensure `xgb_model.pkl` is in the same directory as the `.exe`. The system will still work using the built-in heuristic classifier.

"Analysis seems slow"

Cause: Large FITS files (500+ slit positions) with Monte Carlo error estimation can take 30–90 seconds.

Fix: This is normal. Watch the Live Console for real-time progress.

Solar Intelligence System — Automated Spectropolarimetric Analysis for the Next Generation of Solar Physics.

DOCUMENT 2 OF 2

Developer Version Manual PYTHON SOURCE

For developers, researchers, and technical integrators.

1. Overview

The **SIS Developer Version** provides full access to the Python source code powering the Hinode SP spectropolarimetric analysis pipeline. This version is intended for users who need to inspect, modify, or extend the system.

The Developer Version consists of two Python files:

FILE	ROLE	LINES
`NEW_solar_analyzer_SIS_v8_fixed.py`	**Backend** — physics, AI classification, visualization, report generation	~3,300
`sis_gui_app_v2.py`	**Frontend** — PySide6 GUI with dark theme, live console, file explorer	~960

The GUI imports and calls the backend's `analyze_fits_file()` function on a background thread. Neither file needs modification for standard analysis.

2. Requirements & Libraries

2.1 Python Version

REQUIREMENT	VERSION
Python	**3.10 or newer** (tested on 3.12 and 3.14)

> **IMPORTANT**
> Python 3.9 and below are not supported due to type-hint syntax (`X | None`) used throughout the codebase.

2.2 Required Libraries

LIBRARY	PURPOSE
numpy	Numerical array operations — the foundation of all physics calculations
pandas	DataFrame management — stores and exports per-pixel results to CSV
astropy	Reads FITS files — the standard astronomical data format from Hinode
scipy	Signal processing (smoothing, peak detection, FFT cross-correlation)
matplotlib	Generates all plots, including the 5-panel Master Dashboard
seaborn	Optional styling enhancement for statistical plots
xgboost	Pre-trained ML classifier for the AI triage phase
joblib	Loads the serialized XGBoost model from `xgb_model.pkl`
google-genai	Google Gemini API client — generates the AI scientific report
PySide6	Qt6 GUI framework — powers the desktop application interface

2.3 Optional Files

FILE	PURPOSE
`xgb_model.pkl`	Pre-trained XGBoost classifier. If absent, the system falls back to a rule-based heuristic.
`logo.png`	Optional startup logo. If absent, a styled text header is shown instead.

3. Installation & Startup

Step 1 — Extract the Source Files

```
unzip main_files_for_manual_start.zip
cd main_files_for_manual_start
```

You should see:

```
main_files_for_manual_start/
├── NEW_solar_analyzer_SIS_v8_fixed.py   ← Backend engine
└── sis_gui_app_v2.py                    ← GUI application
```

If you have `xgb_model.pkl`, place it in the same directory.

Step 2 — Create a Virtual Environment (Recommended)

```
python -m venv sis_env

# On Windows:
sis_env\Scripts\activate

# On macOS / Linux:
source sis_env/bin/activate
```

Step 3 — Install Dependencies

```
pip install numpy pandas astropy scipy matplotlib seaborn xgboost joblib google-
```

Or, if a `requirements.txt` file is provided:

```
pip install -r requirements.txt
```

Step 4 — Set the Gemini API Key (Optional)

```
# On Windows (PowerShell):
$env:GEMINI_API_KEY = "your-api-key-here"

# On macOS / Linux:
export GEMINI_API_KEY="your-api-key-here"
```

> **NOTE**
> A default key is hardcoded in the backend for development convenience. For production or shared environments, override it with your own key using the environment variable above.

Step 5 — Launch the GUI

```
python sis_gui_app_v2.py
```

Alternative: Run the Backend Directly (No GUI)

```
# Command-line execution:
python NEW_solar_analyzer_SIS_v8_fixed.py --fits your_data.fits --out my_results
```

Or import it in your own Python script:

```
from NEW_solar_analyzer_SIS_v8_fixed import analyze_fits_file
```

```
df = analyze_fits_file("your_data.fits", output_prefix="my_results")
print(df.head())
```

4. System Architecture

The SIS pipeline has three distinct phases that execute sequentially:

4.1 Phase 1 — XGBoost AI Triage (Classification)

What it does: Before any physics calculations begin, every pixel in the FITS data cube is classified into one of three routing categories.

How it works:

1. For each slit position, the system extracts a compact **16-dimensional feature vector** — four statistical measures (max amplitude, standard deviation, mean, signal-to-noise ratio) for each of the four Stokes parameters (I, Q, U, V).
2. All feature vectors are stacked into a single matrix and passed to the **XGBoost model** in one vectorized `model.predict()` call. This classifies the entire dataset in milliseconds.
3. Each pixel receives one of three labels: "**Clear**" (route to Sigma-V), "**Noisy**" (route to WFA), or "**Anomaly**" (flag and archive).

> **FALLBACK**
> If `xgb_model.pkl` is missing or XGBoost is not installed, the system falls back to a deterministic rule-based heuristic classifier that uses peak detection, SNR thresholds, and skewness analysis. Results are functionally equivalent.

Key code location: `SolarIntelligenceSystem` class

4.2 Phase 2 — Physics Engine (The "Fantastic Four")

Based on the Phase 1 routing labels, each pixel is dispatched to the appropriate physics solver:

PARAMETER	METHOD	WHAT IT MEASURES
LOS Magnetic Field	Sigma-V / WFA	Magnetic field strength along line of sight (Gauss)
Transverse Field	WFA linear polarization	Magnetic field perpendicular to line of sight
Plasma Velocity	Centre-of-Gravity Doppler	Speed of solar plasma toward/away from Earth (km/s)
Temperature Profile	Eddington-Barbier / Planck	Brightness temperature at 5 atmospheric depth points (K)
Turbulence	Non-thermal FWHM subtraction	Chaotic plasma motion beyond thermal broadening (km/s)

Monte Carlo error estimation runs for every pixel where a magnetic field was detected, producing confidence intervals on the B-field measurement.

4.3 Phase 3 — Gemini AI Reporter

After all physics is complete and the dashboard is generated, the system compiles summary statistics and sends a **single API call** to Google Gemini 2.5 Flash. The response is saved to `*_AI_SCIENTIFIC_REPORT.txt`.

If the API key is missing, the network is down, or `google-genai` is not installed, the report is simply skipped. All other outputs are unaffected.

4.4 GUI ↔ Backend Connection

1. The user clicks **"LAUNCH SIS ANALYSIS"** in the Control Center tab.
2. The GUI creates a `QThread` and moves an `AnalysisWorker` onto it.
3. The worker calls `analyze_fits_file()` — all `print()` output is captured via `sys.stdout` redirection and displayed in the live console.
4. When the worker emits its `finished` signal, the GUI loads the dashboard, report, refreshes the file explorer, and switches to the dashboard tab.

All analysis runs off the main thread, so the GUI remains responsive throughout processing.

5. Output File Reference

FILE PATTERN	CONTENT
`*_results.csv`	Full per-pixel measurements (B-field, velocity, temperature, turbulence)
`*_SIS_v3_Master_Dashboard.png`	Combined 5-panel visualization
`*_AI_SCIENTIFIC_REPORT.txt`	Gemini-generated scientific interpretation
`*_B_profile.png`	Magnetic field strength along the scan
`*_B_sigma_profile.png`	Monte Carlo B-field uncertainty
`*_LOS_velocity_COG.png`	Doppler velocity scatter plot
`*_turbulence_profile.png`	Non-thermal broadening profile
`*_temperature_stratification.png`	2D brightness temperature heatmap
`*_magnetic_vector_overview.png`	LOS + transverse field components
`*_examples/`	Individual 4-panel diagnostic plots (Stokes I, Q, U, V)
`*_anomalies/`	Archived anomalous profiles as `.npy` files

6. Troubleshooting

"ModuleNotFoundError: No module named 'PySide6'"

Cause: Dependencies are not installed.

```
pip install PySide6
```

Or install all dependencies at once:

```
pip install numpy pandas astropy scipy matplotlib seaborn xgboost joblib google-
```

"SyntaxError: X | None"

Cause: You are using Python 3.9 or older.

```
python --version
```

If it shows 3.9 or below, install Python 3.10+ from python.org.

"XGBoost model not found at 'xgb_model.pkl'"

Cause: The pre-trained model file is not in the working directory.

Fix: Place `xgb_model.pkl` in the same folder as the Python scripts. The heuristic fallback activates automatically.

"Gemini report generation failed" / "GEMINI_API_KEY not set"

Cause: The API key is missing or `google-genai` is not installed.

```
pip install google-genai
export GEMINI_API_KEY="your-key-here"
```

> **NOTE**
> This error does not affect the physics analysis, CSV output, or dashboard generation. The AI report is an optional enhancement.

"The GUI freezes during analysis"

Cause: An unhandled exception may have killed the worker thread. Check the Live Console for errors. Try running the backend directly:

```
python NEW_solar_analyzer_SIS_v8_fixed.py --fits your_data.fits --out debug_test
```

"FITS data must be 3D" / "Unexpected FITS shape"

Cause: The input file is not a standard Hinode SOT/SP FITS file.

Fix: SIS expects a 3D data cube with shape `(4, N_slits, N_wavelengths)`. Verify with:

```
from astropy.io import fits
hdul = fits.open("your_file.fits")
print(hdul[0].data.shape)
```

7. Extending the System

Adding a New Physics Parameter

1. Write a standalone function (e.g., `compute_my_parameter()`) following the pattern of `compute_los_velocity_cog()`.
2. Call it inside the `# Step 2b: "Fantastic Four"` block in the main loop.
3. Add the result to the `row` dictionary with a descriptive column name.
4. The new column will automatically appear in the CSV output.

Training a New XGBoost Model

```
import xgboost as xgb
import joblib

model = xgb.XGBClassifier(n_estimators=200, max_depth=8, random_state=42)
model.fit(X_train, y_train)  # X: (N, 16) features, y: class codes (0, 1, 2)
joblib.dump(model, "xgb_model.pkl")
```

Place the new `xgb_model.pkl` in the working directory and the system will use it automatically on the next run.

Solar Intelligence System — Automated Spectropolarimetric Analysis for the Next Generation of Solar Physics.